Audio Synthesizer

Pászti Tibor Miklós

November 2, 2026

Abstract

The main goal for this paper is to describe the process of synthesizing music from a MIDI file, meaning that I read the given tracks and thus notes of the song beign supplied to the program and synthesize the audio based on what oscillators and effects are present. The end product will be a PCM file that could be converted to any other audio format needed.

In this paper I will be talking about how to synthesize sound, alter it with effects, how to build these effects and how to visualize the synthesized audio in many forms.

The examples in this paper will be shown in the c programing language and for the visual representation I will be using the SDL3 library.

Although I will be talking about some basic concepts I won't be writing about these concept in an educational manner. I will be citing the relevant sections of my sources that I will be using so for further or more indepth information I recommend reading the sources I will be using relating to audio synthesis. All the sources will be shown at the bibliography section of the paper and they will also be cited at the relevant sections.

Contents

1 Basic Workings: 2
- 1.1 Waveforms and Their Representation: . . . 2
  - 1.1.1 Time-domain Representation: . . . 2
  - 1.1.2 Frequency-domain Representation: . . . 3
- 1.2 Waveform Table: . . . 4
- 1.3 Phase: . . . 5

Bibliography 5

1 Basic Workings:

In the most basic sense my synthesizer will go through the following steps:

1. Read the MIDI file and store it in a fitting structure.
2. Go through each track and it's notes and calculate the note's amplitude at a given phase for the selected wavetable.
3. If more than one note is active, then get the avarage of the amplitudes.
4. Write every amplitude into a PCM audio file for playback.
5. Play the audio with the SDL3 library's mixer.

This describes a very basic synthesizer program, which I will use to explain the core workings of a synthesizer, mainly speaking about the second and third points talking about the calculating of the amplitudes and how to combine these amplitudes.

1.1 Waveforms and Their Representation:

A waveform is any series of values with a repeating pattern. One occurance of this pattern is called a *cycle* and the length of this cycle is the *wavelength* or *period*. As the period increases, the frequency of cycles per second decreases and the sound becomes "lower".

> **Note**
>
> Frequency (**Hz**) states the number of cycles per second.

1.1.1 Time-domain Representation:

First I would like to state what exactly does it mean when a wave goes above or below zero.

The sound people hear is actually the air pressure changing over a period of time, so when a speaker is "moving" it is actually just changing the air pressure according to the analog signal beign supplied to it.

The **time-domain representation** is just a graph of air pressure over time.

To create an audible sound the frequency needs to be in the range of 20 Hz to 20 KHz. Any higher or lower and the human ear can't process it.

> **Note**
>
> The figures will be using a sample size of 4096 because this will be the size of a waveform table as stated in the future sections.

> **TODO**
>
> HREF TO THE RELEVANT SECTION.

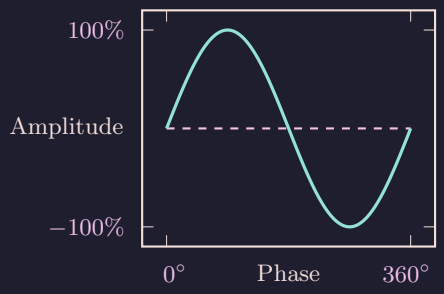


Figure 1: Sine Time-domain Representation

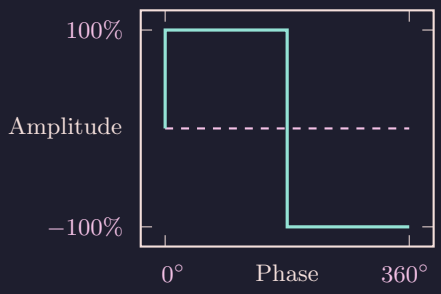


Figure 2: Square Time-domain Representation

When the amplitude is negative the air pressure is decreasing and the same applies in the opposite manner. In the purest sense the amplitude of a sound is describing the changes in air pressure.

> **Note**
>
> As the topic moves to more abstract concepts the meaning of amplitude will change to the volume level of a sound as a range from 0 to 1.

1.1.2 Frequency-domain Representation:

Besides the *fundamental frequency* beign supplied to the oscillator, depending on the waveform there may be other *overtones* or *harmonics* present.

For example the sine wave is said to be pure for one reason because it has no overtones other than the fundamental frequency. The **frequency-domain representation** exists to show these overtones.

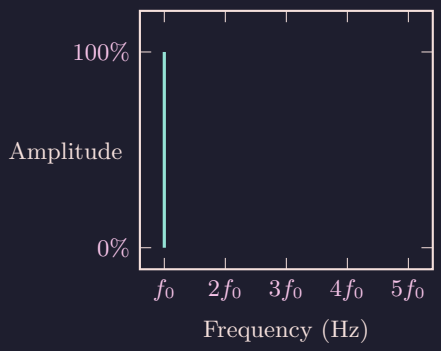


Figure 3: Sine Frequency-domain Representation

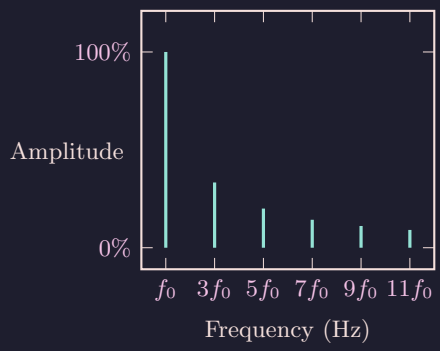


Figure 4: Square Frequency-domain Representation

The frequency-domain representation is achieved by using the fourier transform on the given audio which will split it into a sum of sine waves with different amplitudes. This can be seen as illustrated in Figure 4.

> **Note**
>
> The square waveforms frequency-domain representation is not in it's full display to save space.

> **Further Information**
>
> For further information read the following source material [Roa96, pp. 14–18].

1.2 Waveform Table:

As the number of notes and tracks grows so will the need for computational power and what does every note has to do?

Calculate a given number of trigonometric functions or even generate a bunch of random numbers if talking about a white noise sound. To get around this I will be using a waveform table, meaning that I will be storing the values of a given waveform in a table with a sample size of 4096.

This way I will be able to just index the current value of a sine function for example. This approach also lets the user draw their own waveform tables afterwhich I can store it in a file and read it if the user wants to play that custom waveform.

$$phase_index = \mathrm{mod}_L(previous_phase + increment)$$

$$output = amplitude * wavetable[phase_index]$$

> **Note**
>
> The mod operation is needed to not overindex the waveform table.

1.3 Phase:

The phase is what tells the oscillator how far along the waveform is in the time-domain representation and so I will have to increment the phase for each note independently.

As seen in the previous section I will be using the phase to index the waveform table but first the increment needs to calculated in the following way:

$$increment = \frac{L * frequency}{samplingFrequency}$$

> **Note**
>
> With the table size beign a constant 4096 and the sampling frequency or samplerate beign a constant 44.1KHz, this is not that big of a deal.

Due to understandable reasons this won't be a complete match to an actually calculated waveform and so slight ditrostions may occur. This is called *table-lookup noise*.

As of now I hope this won't be too audible and I can ignore this slight problem.

Bibliography

[Roa96] Curtis Roads. *The Computer Music Tutorial*. The MIT Press, 1996. ISBN: 9780262181587.